

Competitive Programming

Section 46 Editorial: Rerooting and Advanced Tree DP

Competitive Programming Course

2026-07-12

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Sum Distances	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity and implementation notes	6
2.5	Full C++ solution	6
2.6	Full Python solution	6
3	B. Farthest Node Distance	7
3.1	Restatement	7
3.2	Observations and algorithm	7
3.3	Correctness proof	7
3.4	Complexity and implementation notes	7
3.5	Full C++ solution	8
3.6	Full Python solution	8
4	C. Paths Through Node	9
4.1	Restatement	9
4.2	Observations and algorithm	9
4.3	Correctness proof	9
4.4	Complexity and implementation notes	10
4.5	Full C++ solution	10
4.6	Full Python solution	10
5	D. Forced Independent Set	11
5.1	Restatement	11
5.2	Constraints and traps	11
5.3	Observations and trace	11
5.4	Algorithm	12
5.5	Correctness proof	12
5.6	Complexity and implementation notes	12
5.7	Full C++ solution	12
5.8	Full Python solution	14
5.9	Common wrong answers	15
5.10	Tests that matter	15
6	Testing discussion	16

1 Editorial overview

1.1 Problem-specific proof map

A. Sum Distances. First compute the root answer and subtree sizes. Moving the root across parent-child edge (u, v) removes $\text{size}[v]$ near vertices and adds $n - \text{size}[v]$ far vertices, giving $\text{ans}[v] = \text{ans}[u] + n - 2 * \text{size}[v]$.

B. Farthest Node Distance. The farthest distance through a child uses that child's downward value; the farthest outside it uses the best sibling or parent value. Prefix/suffix maxima exclude the child without double counting.

C. Paths Through Node. Removing a node splits the tree into component sizes. Every path through it chooses endpoints from two different components (or the node), so summing cross products counts each unordered pair exactly once.

1.2 Detailed solution guide

A. Sum of Distances from Every Root. First root the tree at 1. A postorder DFS computes $\text{sub}[v]$ and $\text{down}[v]$, where $\text{down}[v]$ is the sum of distances from v to vertices in its subtree. The answer for root 1 is $\text{down}[1]$. When rerooting across edge $v - c$, the $\text{sub}[c]$ vertices inside c 's subtree become one step closer, and the other $n - \text{sub}[c]$ vertices become one step farther. Therefore $\text{ans}[c] = \text{ans}[v] - \text{sub}[c] + (n - \text{sub}[c])$.

B. Tree Diameter. Run a DFS/BFS from any vertex to find a farthest vertex a . Run again from a ; the farthest vertex b gives the diameter distance. The first search reaches an endpoint of some diameter because any path can be extended toward a farthest vertex, and the second search measures the longest path starting from that endpoint.

C. Pairs Separated by Removing a Vertex. Removing vertex v splits the tree into components: one for each child subtree and, unless v is the root, one parent-side component of size $n - \text{sub}[v]$. Pairs whose path passes through v are exactly pairs chosen from two different components. Summing component sizes seen so far gives $\text{sum} += \text{seen} * \text{comp_size}$, then $\text{seen} += \text{comp_size}$.

Common pitfalls. In A, the reroot formula changes distances for all vertices, not just descendants. In C, decide whether pairs are ordered or unordered before multiplying by two; the local task uses unordered pairs.

Each task asks for every-node information on a tree.

- A first computes subtree sizes and the answer for root 1; moving the root across edge $u - v$ changes distances by $-\text{sub}[v] + (n - \text{sub}[v])$.
- B uses the diameter endpoint theorem: the farthest node from any vertex is one of the two diameter endpoints.
- C removes each node conceptually and counts pairs of endpoints lying in different resulting components.

The proof technique is contribution accounting: decide how one edge/root move changes every affected node or pair exactly once.

2 A. Sum Distances

2.1 Restatement

For every node, print the sum of distances to all other nodes.

2.2 Observations and algorithm

Root the tree at vertex 1. Let $\text{sub}[v]$ be the number of vertices in the rooted subtree of v , and let $\text{down}[v]$ be the sum of distances from v to vertices in that subtree. In a postorder traversal, every child to gives

```
sub[v] += sub[to]
down[v] += down[to] + sub[to]
```

The added $\text{sub}[to]$ accounts for the one extra edge from v to every vertex in that child subtree. Thus $\text{down}[1]$ is the answer for root 1.

For a parent-child edge v - to , reroot from v to to . The $\text{sub}[to]$ vertices inside the child subtree become one edge closer, while the other $n - \text{sub}[to]$ vertices become one edge farther:

```
answer[to] = answer[v] - sub[to] + (n - sub[to])
```

Run this transition in a second preorder traversal. On the three-vertex tree with edges 1-2 and 1-3, $\text{answer}[1]=2$ and $\text{sub}[2]=1$, so $\text{answer}[2]=2-1+(3-1)=3$, matching distances 1+2 from vertex 2.

2.3 Correctness proof

Lemma 1. After the postorder traversal, $\text{sub}[v]$ is the size of the rooted subtree of v , and $\text{down}[v]$ is the sum of distances from v to vertices in that subtree.

Proof. Both statements hold at a leaf. Child subtrees are disjoint; adding their sizes counts every descendant once. Every distance from v into a child subtree is the child's distance plus one, so the displayed recurrence adds exactly all subtree distances. Induction up the tree proves the lemma.

Lemma 2. If to is a child of v , the reroot formula computes the sum of distances from to to all vertices.

Proof. Moving across v - to decreases the distance to each of the $\text{sub}[to]$ inside vertices by one and increases the distance to each remaining vertex by one. Applying these two changes to $\text{answer}[v]$ gives the formula.

Theorem. Every printed value is the required sum of distances.

Proof. Lemma 1 gives the root answer. The preorder traversal reaches every other vertex through one parent edge, and Lemma 2 transfers a correct answer across that edge. Induction over this traversal proves every answer correct.

2.4 Complexity and implementation notes

Both traversals inspect every edge a constant number of times: $O(n)$ time and $O(n)$ memory. Distances sums can be quadratic, so use `long long` in C++. On a chain, recursive DFS can exceed the call stack; the local references raise Python's recursion limit, but an iterative traversal is also a sound choice. Test `n=1`, a chain, and a star.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

2.6 Full Python solution

```
import sys
sys.setrecursionlimit(1_000_000)
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; g=[[] for _ in range(n)]; idx=1
    for _ in range(n-1):
        a,b=data[idx]-1,data[idx+1]-1; idx+=2; g[a].append(b); g[b].append(a)
    sub=[1]*n; down=[0]*n
    def dfs(u,p):
        for v in g[u]:
            if v!=p: dfs(v,u); sub[u]+=sub[v]; down[u]+=down[v]+sub[v]
    ans=[0]*n
    def reroot(u,p):
        for v in g[u]:
            if v!=p:
                ans[v]=ans[u]+n-2*sub[v]; reroot(v,u)
    dfs(0,-1); ans[0]=down[0]; reroot(0,-1); print(" ".join(map(str,ans)))
if __name__=="__main__": main()
```

3 B. Farthest Node Distance

3.1 Restatement

For every node, print its distance to the farthest node.

3.2 Observations and algorithm

Do not reroot a DP here. In a tree, choose any start vertex and run BFS to a farthest vertex **a**. Run BFS from **a** to a farthest vertex **b**; **a** and **b** are endpoints of a diameter. A third and final BFS from **b** gives arrays `dist_a` and `dist_b`. For each vertex **v**, print

```
max(dist_a[v], dist_b[v]).
```

For example, on a path 1-2-3-4, the endpoints are 1 and 4. At vertex 2, the two distances are 1 and 2; its farthest-node distance is 2.

3.3 Correctness proof

Lemma 1. The two sweeps find diameter endpoints **a** and **b**.

Proof. In a tree, a farthest vertex from any start is an endpoint of some diameter: otherwise its unique path toward a diameter could be extended to a vertex farther from the start. Starting from such an endpoint, a farthest vertex is the opposite endpoint of a longest path, so **a-b** is a diameter.

Lemma 2. For every vertex **v**, at least one farthest vertex from **v** is **a** or **b**.

Proof. Project the paths from **v** to a farthest vertex and to the diameter path **a-b**. If that farthest path leaves the diameter at some attachment vertex, replacing its endpoint by the farther of **a** and **b** cannot shorten the path from **v**; otherwise the diameter could be extended through that endpoint. Hence the eccentricity of **v** is the larger endpoint distance.

Theorem. The algorithm prints the farthest-node distance for every vertex.

Proof. Lemma 1 supplies a diameter. Lemma 2 says the maximum of the two computed BFS distances equals the maximum distance to any vertex, exactly the requested value.

3.4 Complexity and implementation notes

Each BFS is $O(n)$ on a tree, so the total time and memory are $O(n)$. Keep a separate visited/distance array for every BFS. Test one vertex, a chain, a star, and a tree with several equally long diameter candidates.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<int>bfs(int s,vector<vector<int>>&g){vector<int>d(g.size(),-1);c
```

3.6 Full Python solution

```
import sys,collections
def bfs(s,g):
    d=[-1]*len(g); d[s]=0; q=collections.deque([s])
    while q:
        u=q.popleft()
        for v in g[u]:
            if d[v]<0: d[v]=d[u]+1; q.append(v)
    return d
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; g=[[] for _ in range(n)]; idx=1
    for _ in range(n-1):
        a,b=data[idx]-1,data[idx+1]-1; idx+=2; g[a].append(b); g[b].append(a)
    d0=bfs(0,g); a=max(range(n),key=d0.__getitem__); da=bfs(a,g); b=max(range(n),key=da.__ge
    print(" ".join(str(max(da[i],db[i])) for i in range(n)))
if __name__=="__main__": main()
```

4 C. Paths Through Node

4.1 Restatement

For every node, count unordered pairs of distinct nodes whose path passes through it.

4.2 Observations and algorithm

Root once at 1 and compute $\text{sub}[v]$. After deleting v , each child to forms a component of size $\text{sub}[to]$; if v is not the root, all vertices above it form one component of size $n - \text{sub}[v]$. A pair of distinct vertices has a path through v exactly when it is **not** entirely inside one of those components. Pairs with v as an endpoint are included automatically.

Therefore start from all $n \text{ choose } 2$ unordered pairs and subtract the pairs inside each component:

```
answer[v] = n choose 2 - sum over components c of (size[c] choose 2).
```

For a path of five vertices, deleting the middle vertex leaves components of sizes two and two. The answer there is $10 - 1 - 1 = 8$: the two pairs inside one side do not pass through the middle; every other pair does.

4.3 Correctness proof

Lemma 1. The components after deleting v have exactly the child-subtree sizes and the optional parent-side size described above.

Proof. Removing v cuts its edge to each child, isolating that entire child subtree. All remaining vertices outside the subtree are connected via the parent direction. These sets are disjoint and cover every vertex other than v .

Lemma 2. A pair of distinct vertices whose endpoints are not v has a path through v iff its endpoints lie in different components after deleting v .

Proof. A tree has one simple path between two vertices. It uses v iff deleting v separates its endpoints. The endpoints are separated exactly when they lie in different resulting components.

Theorem. The formula prints the requested count for every vertex.

Proof. There are $n \text{ choose } 2$ pairs in total. By Lemma 2, precisely the pairs contained inside one component fail to pass through v ; these sets are disjoint. Subtracting their counts retains every qualifying pair, including pairs with endpoint v , exactly once.

4.4 Complexity and implementation notes

Computing subtree sizes and scanning all adjacency lists are $O(n)$ time and $O(n)$ memory. Use `long long` for pair counts. For each neighbour, use `sub[child]` only when it is a child; the parent-side component is the single value `n-sub[v]`. Test a singleton, a chain, a star, and a non-root branching vertex.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

4.6 Full Python solution

```
import sys
sys.setrecursionlimit(1_000_000)
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; g=[[] for _ in range(n)]; idx=1
    for _ in range(n-1):
        a,b=data[idx]-1,data[idx+1]-1; idx+=2; g[a].append(b); g[b].append(a)
    parent=[-1]*n; order=[0]
    for u in order:
        for v in g[u]:
            if v!=parent[u]: parent[v]=u; order.append(v)
    sub=[1]*n
    for u in reversed(order[1:]): sub[parent[u]]+=sub[u]
    total=n*(n-1)//2; out=[]
    for u in range(n):
        bad=0
        for v in g[u]:
            s=sub[v] if parent[v]==u else n-sub[u]
            bad+=s*(s-1)//2
        out.append(str(total-bad))
    print(" ".join(out))
if __name__=="__main__": main()
```

5 D. Forced Independent Set

5.1 Restatement

For every vertex v , find the largest independent set that must include v . An independent set contains no edge whose two endpoints are both selected. Running a tree DP from scratch for each forced vertex would be quadratic.

5.2 Constraints and traps

With $n=200000$, one full DP per forced vertex is too slow. The state must say whether its current vertex is selected: that choice constrains every neighbour.

5.3 Observations and trace

Root the tree. For each vertex v , first compute values for its subtree only:

```
down_with[v] = best size in v's subtree when v is selected
down_without[v] = best size in v's subtree when v is not selected
```

For every child to , selecting v forces to not to be selected; leaving v unselected lets to choose its better state. Thus:

```
down_with[v] = 1 + sum(down_without[to])
down_without[v] = sum(max(down_with[to], down_without[to]))
```

These values omit the part above v . Store matching outside contributions:

```
outside_with[v] = best contribution outside v's subtree if v is selected
outside_without[v] = best contribution outside v's subtree if v is not selected
```

Both are zero at the root. To pass information from a parent v to a child to , remove to from the parent's child sums:

```
free_siblings = down_without[v] - max(down_with[to], down_without[to])
without_siblings = down_with[v] - 1 - down_without[to]

outside_with[to] = outside_without[v] + free_siblings
outside_without[to] = max(
    outside_without[v] + free_siblings,
    outside_with[v] + 1 + without_siblings
)
```

The first transition makes the parent unselected because `to` is selected. In the second, the parent may be selected or unselected. The requested answer is `down_with[v] + outside_with[v]`. In the sample, forcing vertex 3 excludes 1, 4, and 5; only vertex 2 can join it, so its answer is 2.

5.4 Algorithm

```
root the tree and record parent plus traversal order
process the order backwards to compute down_with and down_without
process the order forwards to compute outside_with and outside_without
for every v, print down_with[v] + outside_with[v]
```

5.5 Correctness proof

Lemma 1. `down_with[v]` and `down_without[v]` are optimal independent-set sizes in v 's subtree under their stated conditions.

Proof. At a leaf the values are one and zero. Child subtrees are disjoint. If v is selected, every child must be unselected; if v is unselected, every child may independently use its better state. The two recurrences therefore combine exactly the feasible optimal child choices. Induction proves the lemma.

Lemma 2. `outside_with[v]` and `outside_without[v]` are optimal contributions outside v 's subtree under their stated conditions.

Proof. Both values are zero at the root. For a parent-child edge, the transition enumerates the only two states of the parent. It combines the already optimal outside contribution with disjoint sibling subtrees, excluding the child itself. Induction in preorder proves the lemma.

Theorem. The printed value for every vertex v is the largest independent set size among sets that contain v .

Proof. Lemma 1 gives the optimal selected set inside v 's subtree and Lemma 2 gives the compatible optimum outside it. The regions are disjoint and their sum selects v ; every feasible global set splits into the same two regions. Hence the sum is feasible and optimal.

5.6 Complexity and implementation notes

Both passes inspect every edge a constant number of times: $O(n)$ time and $O(n)$ memory. Use an iterative traversal so a 200000-vertex chain cannot overflow recursion. Recomputing all sibling contributions for every child would be quadratic; subtract the child's known contribution from the parent sum.

5.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<vector<int>> graph(n);
    for (int edge = 0; edge + 1 < n; ++edge) {
        int left, right;
        cin >> left >> right;
        --left;
        --right;
        graph[left].push_back(right);
        graph[right].push_back(left);
    }

    vector<int> parent(n, -1), order{0};
    for (int index = 0; index < (int)order.size(); ++index) {
        int vertex = order[index];
        for (int neighbor : graph[vertex]) {
            if (neighbor != parent[vertex]) {
                parent[neighbor] = vertex;
                order.push_back(neighbor);
            }
        }
    }

    vector<int> down_without(n), down_with(n, 1);
    for (int index = n - 1; index >= 0; --index) {
        int vertex = order[index];
        for (int child : graph[vertex]) {
            if (parent[child] == vertex) {
                down_without[vertex] += max(down_without[child], down_with[child]);
                down_with[vertex] += down_without[child];
            }
        }
    }

    vector<int> outside_without(n), outside_with(n), answer(n);
    for (int vertex : order) {
        answer[vertex] = down_with[vertex] + outside_with[vertex];
        for (int child : graph[vertex]) {
            if (parent[child] != vertex) continue;
            int siblings_free = down_without[vertex] - max(down_without[child], down_with[child]);
            int siblings_without = down_with[vertex] - 1 - down_without[child];
            outside_with[child] = outside_without[vertex] + siblings_free;
            outside_without[child] = max(
                outside_without[vertex] + siblings_free,

```

```

        outside_with[vertex] + 1 + siblings_without
    );
}

for (int vertex = 0; vertex < n; ++vertex) {
    cout << answer[vertex] << (vertex + 1 == n ? '\n' : ' ');
}
}

```

5.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    graph = [[] for _ in range(n)]
    for index in range(1, 2 * n - 1, 2):
        left, right = data[index] - 1, data[index + 1] - 1
        graph[left].append(right)
        graph[right].append(left)

    parent = [-1] * n
    order = [0]
    for vertex in order:
        for neighbor in graph[vertex]:
            if neighbor != parent[vertex]:
                parent[neighbor] = vertex
                order.append(neighbor)

    down_without = [0] * n
    down_with = [1] * n
    for vertex in reversed(order):
        for neighbor in graph[vertex]:
            if parent[neighbor] == vertex:
                down_without[vertex] += max(down_without[neighbor], down_with[neighbor])
                down_with[vertex] += down_without[neighbor]

    outside_without = [0] * n
    outside_with = [0] * n
    answer = [0] * n
    for vertex in order:
        answer[vertex] = down_with[vertex] + outside_with[vertex]
        for child in graph[vertex]:

```

```

        if parent[child] != vertex:
            continue
        siblings_free = down_without[vertex] - max(down_without[child], down_with[child])
        siblings_without = down_with[vertex] - 1 - down_without[child]
        outside_with[child] = outside_without[vertex] + siblings_free
        outside_without[child] = max(
            outside_without[vertex] + siblings_free,
            outside_with[vertex] + 1 + siblings_without,
        )

    print(" ".join(map(str, answer)))

if __name__ == "__main__":
    main()

```

5.9 Common wrong answers

Allowing a selected parent and selected child together.
 Using a parent's full sum without subtracting the current child.
 Combining `outside\_with` with a selected parent across the shared edge.
 Returning the ordinary global maximum instead of forcing the queried vertex.

5.10 Tests that matter

n=1.
 A star, where a forced center and a forced leaf behave very differently.
 A chain, which also checks iterative traversal.
 A branching tree where forcing an internal vertex excludes several neighbours.

6 Testing discussion

For A, compare each answer on tiny trees with a BFS from every start. For B, compare the diameter-endpoint answer with the maximum entry of a BFS distance row. For C, enumerate pairs and test whether their unique path contains the vertex. For D, enumerate all subsets on tiny trees, reject any subset containing both endpoints of an edge, and retain the best size for every included vertex. Then run:

```
CP_TARGET=solution python3 sections/46_rerooting_advanced_tree_dp/check.py
```